CASE STUDY

NEXTGEN Surveillance

Computer Vision / Backend Engineering / Event-Based Alerts

OVERVIEW

NEXTGEN connects camera footage to reviewable face-alert records. Its backend reads RTSP video, compares detected faces with enrolled records, and stores unknown-face events for review through a React dashboard.

THE PROBLEM

Camera monitoring needs a path from visual observation to an event an operator can inspect. The backend brings together frame processing, face comparison, camera context, image capture, and alert storage so detections can be reviewed alongside camera and face records.

MY ROLE

I developed the NEXTGEN backend, covering camera and face APIs, detection processing, face tracking, alert persistence, and streaming endpoints. This server-side work connects visual processing and stored events to the React dashboard through backend interfaces.

WORKFLOW

An operator configures cameras and enrolled faces, then arms detection. Unknown-face events are saved with a timestamp, camera reference, status, and JPEG snapshot. The dashboard retrieves those records and presents active alerts and history for review.

SYSTEM DESIGN

From camera to alert

Face processing and event persistence behind the dashboard

VISUAL INPUT

OpenCV reads RTSP sources in per-camera threads after a connectivity check and timeout. Face detection and embedding generation run inside the backend using dlib and face_recognition.

TRACKING

Detected embeddings are compared with enrolled face records. Local tracking, a process-wide tracker, distance thresholds, and per-track cooldowns reduce repeated alert creation. Some cross-camera matches reuse or move an existing unknown-face alert.

STORAGE

FastAPI exposes camera, face, alert, and detection-control routes. SQLAlchemy persists records in SQLite. Alerts store event metadata and a base64 JPEG camera frame; face records store an image and embedding. The dashboard polls the alert API every five seconds.

EVENT PATH

RTSP camera > OpenCV frame capture
Face detection > Embedding comparison and tracking
Unknown-face event > SQLite alert and JPEG snapshot
REST API > Dashboard review and alert history

A review action can enroll the alert image as a face record and mark the alert inactive.

STREAMING

The selected backend implements aiortc WebRTC signaling, FFmpeg-based HLS, and HLS fallback when WebRTC startup fails. These viewing paths are separate from alert persistence. Live playback remains unverified across backend and web configuration.

ENGINEERING

Decisions and outcomes

Backend capabilities and the limits of the current evidence

REPEAT EVENTS

A face can appear in many consecutive frames or move between cameras. Tracking, embedding association, and cooldowns address that repetition before new events are written. The approach uses in-memory state and heuristic thresholds; measured recognition accuracy is not established.

FAILURES

Camera readiness checks avoid starting processing on an unavailable input. Failed alert writes are rolled back and logged. Processing uses local threads and synchronous database commits, so durable retries and automatic camera reconnection remain limitations.

OUTCOME

The implementation connects RTSP ingestion, face comparison, alert persistence, and dashboard-facing APIs. It supports camera and face management, batch detection controls, and alert review. Deployment, successful live-camera operation, and performance results have not been independently verified.

TECHNOLOGY

Backend: Python, FastAPI
Vision: OpenCV, dlib, face_recognition
Data: SQLAlchemy, SQLite
Streaming: FFmpeg, HLS, aiortc, WebRTC
Dashboard integration: React, REST polling, HLS.js

EVIDENCE

Documented baseline: backend origin/master at 67e21bf; web main at fad7d87. Key modules include detection_service.py, tracker.py, alert routes, and database models. Backend main is an alternative HLS-only version; streaming contracts and API-base configuration differ between versions.